Übung 2

Aufbau von Programmiersprachen

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Ziel: Kleine Sprachen mit kontextfreien Grammatiken (Typ 2) beschreiben, Wörter herleiten und die zulässige Form eines Programms von seiner Bedeutung unterscheiden.

Arbeitsweise: Nutzen Sie die Folien zu Kapitel 2. Bearbeiten Sie die Aufgaben auf Papier oder digital; Java ist nicht erforderlich.

Reihenfolge: Basic (1-4) festigt die Grundlagen, Combined (5-8) verbindet sie, Challenge (9-10) vertieft den Entwurf eigener Regeln.

Notation: ␣ (in einigen Folien ␣) steht für genau ein Leerzeichen, ↵ für einen Zeilenumbruch. Anführungszeichen begrenzen dargestellte Wörter und gehören nicht dazu.

01 Alphabet und Sprache unterscheiden | Basic

Gegeben sind das Alphabet Σ = {0, 1} und die Sprache L der Binärdarstellungen ohne führende Nullen. Die einzelne 0 ist erlaubt.

a) Prüfen Sie für `ε`, `0`, `101`, `01` und `102` jeweils getrennt: Gehört die Zeichenfolge zu Σ*? Gehört sie zu L? Begründen Sie jede Ablehnung.

b) Seien u = `10` und v = `01`. Bestimmen Sie uv, vu, uε und |uv|. Ist die Konkatenation vertauschbar? Begründen Sie am Beispiel.

c) Betrachten Sie nun das Algo-Alphabet. Bestimmen Sie die Länge von `GIB␣0␣AUS↵`. Erklären Sie, weshalb ␣ und ↵ jeweils kein leeres Wort sind.

02 Syntax und Bedeutung prüfen | Basic

Die Variable `wert` ist deklariert und enthält 4. Ziel ist, ihren Wert zu verdoppeln und das Ergebnis wieder in `wert` zu speichern. Jede Variante startet unabhängig mit 4; jede Zeile endet mit einem Zeilenumbruch.

A
```
SETZE wert AUF wert * 2
```

B
```
SETZE wert AUF wert + 2
```

C
```
SETZE wert wert * 2
```

D
```
SETZE wert AUF wert *
```

a) Welche Varianten sind syntaktisch korrektes Algo? Benennen Sie bei den übrigen die verletzte Schreibregel.

b) Bestimmen Sie für jede syntaktisch korrekte Variante den gespeicherten Wert. Welche erfüllt das Ziel?

c) Korrigieren Sie alle Varianten, die das Ziel nicht erfüllen. Erklären Sie den Unterschied zwischen einem Syntaxfehler und einem gültigen Programm mit falschem Ergebnis.

Basic: Ableitungen und Regelformen

03 Mit einer Grammatik arbeiten | Basic

Gegeben ist $G = (\Sigma, N, P, S)$ mit $\Sigma = \{0, 1\}, N = \{X, R\}$ und $S = X$. Die Produktionsmenge lautet:

$$P = \{ X \to 0,\ X \to 1R,\ R \to 0R,\ R \to 1R,\ R \to \varepsilon \}$$

Jeder Schritt $\vdash$ wendet genau eine Produktion an; $\vdash^*$ erlaubt null oder mehr Schritte.

a) Leiten Sie 0 und 1011 vollständig aus X her. Notieren Sie zu jedem Schritt die verwendete Produktion und zählen Sie die Schritte.

b) Begründen Sie anhand der Regeln, warum 01 und ε nicht zur erzeugten Sprache gehören.

c) Die Zeichenfolge $1R$ ist aus X ableitbar. Gehört sie zu L(G)? Begründen Sie Ihre Antwort und unterscheiden Sie dabei Ableitbarkeit und Sprachzugehörigkeit.

04 Typ 2 von anderen Regelformen abgrenzen | Basic

Für alle vier Grammatiken gelten Σ = {a, b}, N = {S, A, B} und das Startsymbol S. Nicht verwendete Nichtterminale sind zulässig.

Grammatik	Produktionen
G1	$S \to aS$ $S \to b$
G2	$S \to aSS$ $S \to b$
G3	$S \to AB$ $AB \to aB$ $A \to a$ $B \to b$
G4	$S \to AB$ $AB \to A$ $A \to a$ $B \to b$

a) Welche Grammatiken erfüllen die Typ-2-Bedingung? Prüfen Sie alle Regeln und nennen Sie bei einer Ablehnung die entscheidende Produktion.

b) Erklären Sie an G2, warum mehrere Nichtterminale auf der rechten Seite mit Typ 2 vereinbar sind. Leiten Sie das Wort abb her.

c) Ordnen Sie die vier Grammatiken ergänzend nach der höchstmöglichen Typnummer von 0 bis 3 ein. Begründen Sie die Einordnung kurz und nennen Sie bei Typ 3 die lineare Richtung.

Beurteilen Sie die gegebenen Regeln, nicht eine mögliche andere Grammatik für dieselbe Sprache. In dieser Aufgabe gibt es keine ε-Produktionen.

Combined: Sprachen selbst beschreiben

05 Eine Grammatik für Zugriffscodes entwerfen | Combined

Ein Zugriffscode beginnt mit `r` oder `w`. Danach folgt eine nichtleere Folge aus `0` und `1`. Am Ende steht genau ein `#`. Leerzeichen sind nicht erlaubt. Beispiele sind `r0#` und `w101#`.

a) Entwerfen Sie eine Typ-2-Grammatik, die genau diese Codes erzeugt. Geben Sie das vollständige Tupel `G = (Σ, N, P, S)` an. Verwenden Sie einzelne Symbole als Nichtterminale und getrennte Produktionen mit →.

b) Leiten Sie mit Ihrer Grammatik `r0#` und `w101#` her. Ersetzen Sie in jedem Schritt genau eine linke Regelseite.

c) Begründen Sie anhand Ihrer Regeln, warum `r#`, `w012#` und `w10##` ausgeschlossen sind. Prüfen Sie auch, ob beide Anfangsbuchstaben und beliebig lange Bitfolgen möglich sind.

d) Schreiben Sie dieselben Produktionen in BNF mit sprechenden Namen für die Nichtterminale. Markieren Sie das Startsymbol.

06 Boolesche Ausdrücke beschreiben | Combined

Boolesche Ausdrücke bestehen hier aus den Variablen `a`, `b`, `c` sowie `!` (Negation), `&&` (Und) und `||` (Oder). Klammern sind erlaubt, Leerzeichen nicht. Startsymbol ist `<Ausdruck>`. Die Zeichen zwischen den Anführungszeichen werden genauso in das erzeugte Wort übernommen. Die Anführungszeichen selbst werden nicht mitgeschrieben. Zum Beispiel: "||" erzeugt die beiden Zeichen ||.

```
<Ausdruck> ::= <Und> { "||" <Und> }
<Und>      ::= <Faktor> { "&&" <Faktor> }
<Faktor>   ::= "!" <Faktor> | <Atom>
<Atom>     ::= a | b | c | "(" <Ausdruck> ")"
```

a) Entscheiden Sie für `a`, `!a&&b`, `a||b&&c`, `!(a||b)`, `a&&`, `||b` und `a!b`, ob sie zur Sprache gehören. Begründen Sie die ungültigen Fälle.

b) Übertragen Sie die EBNF in eine gleichwertige BNF. Ersetzen Sie die Wiederholungen durch zusätzliche Nichtterminale und Produktionen. Verwenden Sie keine EBNF-Klammern.

c) Leiten Sie `!a||b` vollständig mit Ihrer BNF ab. Kennzeichnen Sie die Schritte, in denen eine ε-Produktion verwendet wird.

d) Zeichnen Sie Syntaxdiagramme für `<Ausdruck>` und `<Faktor>`. Markieren Sie darin die Wege, die Sie bei der Ableitung von `!a||b` verwenden.

Combined: Die Sprache Algo anwenden

07 Ausdrücke und Einzelanweisungen beschreiben | Combined

Die folgenden Regeln sind ein Auszug aus der Algo-Grammatik der Vorlesung. Für a) und b) genügen dieser Auszug und die nachstehenden Hinweise. Verwenden Sie für c) und d) zusätzlich die Vorlesungsfolien zu Zuweisung und Ausgabe. Bezeichner bestehen aus einem oder mehreren Kleinbuchstaben von a bis z. Ganzzahlen haben keine führenden Nullen; `0` ist erlaubt, `-0` nicht.

```
<Wert> ::= <Bezeichner> | <Ganzzahl>
<Ausdruck> ::= <Wert> [ ␣ <Rechenoperator> ␣ <Wert> ]
<Rechenoperator> ::= + | - | * | /
```

Nur ␣ steht hier für ein erzeugtes Leerzeichen. Andere Abstände zwischen Grammatikteilen dienen der Lesbarkeit.

a) Welche der folgenden Wörter sind Ausdrücke in Algo? Begründen Sie jeweils: `x`, `-7`, `x␣+␣1`, `x+1`, `x␣+␣1␣+␣2`, `(x␣+␣1)`, `03`, `-0`.

b) Schreiben Sie die Regel für `<Ausdruck>` in gleichwertiger BNF ohne eckige Klammern.

c) Formulieren Sie die Regeln für `<Zuweisung>` und `<Ausgabe>` in BNF. Geben Sie alle Leerzeichen und den abschließenden Zeilenumbruch ausdrücklich an. Nutzen Sie die Form der Vorlesung.

d) Ist `GIB␣x␣+␣1␣AUS↲` zulässig? Schreiben Sie eine gültige Folge aus Zuweisung und Ausgabe, die x + 1 ausgibt und x unverändert lässt. Eine Hilfsvariable `ergebnis` ist bereits deklariert.

08 Ein Algo Programm schrittweise erzeugen | Combined

Betrachten Sie das folgende Programm. Jede dargestellte Zeile endet mit ↲, auch die letzte. Die Rumpfzeile beginnt mit genau zwei Leerzeichen.

```
DEKLARIERE n
LIES n EIN
SOLANGE n > 0:
  SETZE n AUF n - 1
ENDE SOLANGE
GIB n AUS
```

a) Leiten Sie aus `<Anweisung>` zunächst eine Sequenz mit vier Anweisungen her. Verwenden Sie die Syntaxdiagramme der Vorlesung.

b) Ersetzen Sie die vier Anweisungen durch Deklaration, Eingabe, Iteration und Ausgabe und entwickeln Sie auch den Schleifenrumpf. Lassen Sie `<Bezeichner>`, `<Bedingung>`, `<Ausdruck>` und `<Wert>` zunächst stehen. Kennzeichnen Sie alle Leerzeichen und Zeilenumbrüche.

c) Leiten Sie anschließend die Bedingung `n␣>␣0` vollständig aus `<Bedingung>` her. Notieren Sie jeden einzelnen Ersetzungsschritt.

d) Ergänzen Sie im Rumpf nach der Zuweisung die Anweisung `GIB n AUS`. Welche Grammatikregel ermöglicht das? Welche Zeilen müssen eingerückt werden?

Challenge: Eigene Regeln entwickeln und erweitern

09 Kfz-Kennzeichen beschreiben | Challenge

Für ein vereinfachtes Modell deutscher Kfz-Kennzeichen gilt: Auf ein Ortskürzel mit 1–3 Großbuchstaben folgen ein Bindestrich, 1–2 Großbuchstaben, genau ein Leerzeichen und eine Zahl von 1 bis 9999 ohne führende Nullen. Beispiel: `BN-AB␣123`.

Erlaubte Buchstaben sind A–Z. Im Modell sind alle passenden Buchstabenfolgen zulässig; tatsächliche Ortskürzel, gesperrte Kombinationen und Sonderkennzeichen bleiben unberücksichtigt.

a) Entwerfen Sie eine Typ-2-Grammatik für genau dieses Modell. Geben Sie das vollständige Tupel `G = (Σ, N, P, S)` an. Verwenden Sie ein Symbol je Nichtterminal und → für die Produktionen. Schreiben Sie Alternativen getrennt; Bereiche wie `A...Z` oder `0...9` dürfen Sie abkürzen.

b) Leiten Sie `BN-AB␣1234` und `LEV-A␣1234` vollständig her. Kennzeichnen Sie dabei die eingesetzten Produktionen.

c) Begründen Sie anhand Ihrer Regeln, warum `LEV-AB␣12345`, `BN-A␣012`, `BN-␣12` und `BN-A␣0` ausgeschlossen sind.

10 Algo um eine weitere Schleife erweitern | Challenge

Erweitern Sie Algo um `WIEDERHOLE ... SOLANGE`. Der Rumpf wird mindestens einmal ausgeführt; die Bedingung wird danach geprüft. Solange sie wahr ist, folgt ein weiterer Durchlauf. Beispiel:

```
WIEDERHOLE
  SETZE n AUF n - 1
SOLANGE n > 0
```

Nach `WIEDERHOLE` und nach der `SOLANGE`-Bedingung steht jeweils ein Zeilenumbruch. Es gibt keinen Doppelpunkt und keine zusätzliche Endzeile. Jede Rumpfzeile wird um zwei Leerzeichen eingerückt; die Einrückung wird zusätzlich zur Grammatik festgelegt.

a) Formulieren Sie eine EBNF-Regel für `<Wiederholung>`. Nutzen Sie `<Anweisung>` für den Rumpf und `<Bedingung>` für den Vergleich. Kennzeichnen Sie alle Leerzeichen und Zeilenumbrüche und formulieren Sie die Einrückungsregel.

b) Benennen Sie die bisherige SOLANGE-Regel `<Iteration>` in `<SolangeIteration>` um. Geben Sie eine neue Regel für `<Iteration>` an, die beide Schleifenformen erlaubt. Die übrigen Produktionsregeln bleiben bestehen. Prüfen Sie außerdem, welche neuen Terminalzeichen das Algo-Alphabet benötigt.

c) Zeichnen Sie das Syntaxdiagramm für `<Wiederholung>`. Erklären Sie, warum die Anweisungen im Schleifenrumpf nur einmal im Programm stehen, obwohl sie mehrfach ausgeführt werden können.

d) Schreiben Sie ein vollständiges Programm mit Deklaration, neuer Schleife und Ausgabe. Es soll ganze Zahlen einlesen, bis eine positive Zahl vorliegt, und diese ausgeben. Prüfen Sie die Eingabefolge `0, -2, 3` sowie eine sofortige Eingabe von `3`. Vergleichen Sie den Zeitpunkt der Bedingungsprüfung mit der bisherigen `SOLANGE`-Schleife.